

# JavaScript & TypeScript Essentials

---

## Course Outline

---

## About This Course

---

- **Duration:** 5 days (Monday-Friday, 8:30 am-4:30 pm ET)
- **Delivery:** Instructor-led, with prepared code demos, hands-on activities, and reference documentation.
- **Audience:** Technical professionals and technical managers who develop or oversee web applications and need a solid foundation in JavaScript and TypeScript to improve productivity, code reliability, and team collaboration. Attendees already program: variables, functions, and loops are assumed.
- **Prerequisites:** Basic computer skills, along with experience in HTML and introductory CSS.
- **Platform:** Windows-first; terminal examples use PowerShell. Node.js (current Active LTS) and Visual Studio Code are used throughout.

## What You Will Learn

- Write clean, functional **JavaScript** for dynamic web pages.
  - Apply **regular expression** patterns to validate user input securely.
  - Parse, generate, and manipulate **JSON** data for seamless API integration.
  - Control the **Document Object Model (DOM)** to create interactive user interfaces.
  - Implement **asynchronous** operations using callbacks, promises, and `async / await`.
  - Transition JavaScript code to **TypeScript**, adding static types for early error detection.
  - Structure projects with **modular** JavaScript / TypeScript and manage dependencies.
  - Debug effectively in browsers and Visual Studio Code, employing **linters and formatters** to enforce code standards.
- 

## Section I: Getting Started

---

- A. What is JavaScript?

- B. The ECMAScript standard
- C. JavaScript development tools
- D. Visual Studio Code
- E. Code linters and formatters
- F. Debug JavaScript in the web browser
- G. Debug JavaScript in VS Code

## **Section II: JavaScript Types, Variables & Objects**

---

- A. Add JavaScript to web pages
- B. Statements, variables, and types
- C. Primitives vs. object types
- D. Variable declaration
- E. Basic functions and parameters
- F. Creating and accessing arrays
- G. Creating objects and properties

## **Section III: Code Blocks**

---

- A. Blocks and scope
- B. Using conditional logic
- C. Looping
- D. Error handling

## **Section IV: Working with Built-In Objects and Arrays**

---

- A. The JavaScript standard library
- B. Built-in JavaScript objects
- C. Using string properties and methods
- D. Using number properties and methods
- E. The JavaScript Math object and randomness
- F. JavaScript dates
- G. Maps and sets

## Section V: Browser Object Model

---

- A. Window object
- B. Document object
- C. Location object

## Section VI: Document Object Model (DOM)

---

- A. Registering and reacting to events
- B. Adding and removing DOM nodes
- C. Changing the style of HTML elements

## Section VII: Using JavaScript with Forms

---

- A. Set / get form data
- B. Submit a form with JavaScript
- C. Prevent form submission
- D. Validating form fields
- E. The Form Data object
- F. Submit the Form Data object

## Section VIII: JavaScript Objects

---

- A. Object creation
- B. Object literals
- C. Function constructors
- D. Classes
- E. The value of `this`
- F. Prototype inheritance

## Section IX: Code Organization

---

- A. Multiple script elements
- B. Modular JavaScript
- C. Static modules

- D. Dynamic modules

## Section X: JSON

---

- A. What is JSON syntax?
- B. JSON vs. XML
- C. JSON serialization

## Section XI: Asynchronous Programming with JavaScript

---

- A. The event loop
- B. Single-threaded programming
- C. Closures
- D. Arrow functions
- E. Callbacks
- F. Promises
- G. Async / await
- H. Sequential tasks
- I. Concurrent tasks
- J. Performance developer tools
- K. Timers
- L. Geolocation
- M. Fetch

## Section XII: TypeScript

---

- A. What is TypeScript?
- B. Static vs. dynamic typing
- C. Strong vs. loose typing
- D. Type coercion
- E. Declare variable types
- F. Declare function parameter types
- G. Type aliases
- H. Interfaces
- I. Abstract classes

- J. Generic types
- K. Utility types